

# Operadores no Python

Veja neste conteúdo os diferentes operadores da linguagem Python e como utilizá-los.

**Por que eu devo ler este artigo:** Os operadores são utilizados para construir expressões lógicas, aritméticas, associativas, atribuir valores a variáveis e compará-los. Trata-se de um recurso utilizado nas tarefas mais fundamentais da programação, conforme veremos neste artigo e, portanto, de conhecimento obrigatório para qualquer programador.

Os operadores são usados na construção de expressões, as quais contém um número variado de operandos. Por exemplo, na expressão `a + b`, temos o operador de aritmético `+` e operandos são as variáveis `a` e `b`.

Na linguagem Python temos a seguinte separação entre os diferentes tipos de operadores:

- Operadores aritméticos
- Operadores de atribuição
- Operadores de comparação
- Operadores lógicos
- Operadores de identidade
- Operadores de associação

A seguir veremos cada um deles.

## Operadores aritméticos

Os operadores aritméticos são utilizados na execução de operações matemáticas, tais como a soma e a subtração, por exemplo. Vejamos na **Tabela 1** a lista deles.

| Operador                        | Conceito                                                                       | Exemplo       |
|---------------------------------|--------------------------------------------------------------------------------|---------------|
| + (Adição ou sinal positivo)    | - Realiza a soma entre operandos - Adiciona o sinal de positivo ao número      | - 10 + 7 - +4 |
| - (Subtração ou sinal negativo) | - Realiza a subtração entre operandos - Adiciona o sinal de negativo ao número | - 10 - 7 - -4 |
| * (Multiplicação)               | Realiza a multiplicação entre operandos                                        | 3 * 4         |
| / (Divisão)                     | Realiza a divisão entre operandos                                              | 10 / 5        |
| // (Divisão inteira)            | Realiza a divisão entre operandos e a parte decimal do resultado               | 10 // 6       |
| % (Módulo)                      | Retorna o resto da divisão entre operandos                                     | 4 % 2         |
| ** (Exponenciação)              | Retorna um número elevado a potência de outro                                  | 4 ** 2        |

**Tabela 1.** Operadores aritméticos

O **Código 1** mostra um exemplo com os operadores aritméticos usados no Python.

```
1  numero_1 = 5
2  numero_2 = 2
3
4  soma = numero_1 + numero_2
5  subtracao = numero_1 - numero_2
6  multiplicacao = numero_1 * numero_2
7  divisao = numero_1 / numero_2
8  divisao_inteira = numero_1 // numero_2
9  modulo = numero_1 % numero_2
10 exponenciacao = numero_1 ** numero_2
11
12 print(soma) # 7
13 print(subtracao) # 3
14 print(multiplicacao) # 10
15 print(divisao) # 2.5
16 print(divisao_inteira) # 2
17 print(modulo) # 1
18 print(exponenciacao) # 25
```

**Código 1.** Operadores aritméticos

Uma característica importante a ser observada quando falamos dos operadores matemáticos é a precedência. Essa característica é relativa à ordem da execução deles e acontece quando mais de um operador está presente numa expressão. Segue a precedência dos operadores no Python.



Nos exemplos abaixo usaremos a função `print()` para exibir os resultados das expressões.

1. As expressões contidas em parênteses têm a precedência maior na linguagem Python. Isso permite que uma expressão execute antes de outra. Ex.:

```
1 | print((2 + 5) * 3) # O resultado será 21
```

2. Após os parênteses, o próximo operador com maior precedência é o de exponenciação. Ex.:

```
1 | print( 1 + 5**2 ) # O resultado será 26
```

3. Multiplicação e divisão têm precedência sobre a adição e subtração: como já é conhecido na matemática, divisão e multiplicação são executadas antes das operações de adição e subtração. Ex.:

```
1 | print(5 * 3 + 8) # O resultado será 23
```

4. Ordem de precedência é avaliada da esquerda para a direita. Portanto, após os operadores anteriores, a sequência da execução será da esquerda para a direita. Ex.:

```
1 | print(8 + 5 - 10) # O resultado será 3
```

## Operadores de atribuição

Os operadores de atribuição atribuem valor a uma variável. Na **Tabela 2** temos uma lista desses operadores.

| Operador | Exemplo | Equivalente a |
|----------|---------|---------------|
| =        | x = 1   | x = 1         |
| +=       | x += 1  | x = x + 1     |
| -=       | x -= 1  | x = x - 1     |
| *=       | x *= 1  | x = x * 1     |
| /=       | x /= 1  | x = x / 1     |
| %=       | x %= 1  | x = x % 1     |

**Tabela 2.** Operadores de atribuição

No **Código 2** temos um exemplo do uso de operadores de atribuição.

```
1 | numero_1 = 5
2 |
3 | numero_1 += 5
4 |
5 | print(numero_1) # O resultado será 10
```

**Código 2.** Operadores de atribuição

Acima, vemos que a variável `numero_1` recebe na linha 3 o valor dela mesmo somado a 5. Isso fará com que o resultado impresso na linha 5 seja igual a 10.

## Operadores de comparação

Os operadores de comparação são usados para comparar valores, o que vai retornar `True` ou `False`, dependendo da condição. A seguir, na **Tabela 3** temos exemplos de alguns usados no Python.

| Operador             | Conceito                                      | Exemplo                |
|----------------------|-----------------------------------------------|------------------------|
| >(Maior que)         | Verifica se um valor é maior que outro        | <code>x &gt; 5</code>  |
| <(Menor que)         | Verifica se um valor é menor que outro        | <code>x &lt; 5</code>  |
| ==(Igual a)          | Verifica se um valor é igual a outro          | <code>x == 5</code>    |
| !=(Diferente de)     | Verifica se um valor é diferente de outro     | <code>x != 5</code>    |
| >=(Maior ou igual a) | Verifica se um valor é maior ou igual a outro | <code>x &gt;= 5</code> |
| <=(Menor ou igual a) | Verifica se um valor é menor ou igual a outro | <code>x &lt;= 5</code> |

**Tabela 3.** Operadores de comparação

Usando a comparação que é feita com esses operadores, podemos criar condições para os códigos. No exemplo abaixo, vemos como isso acontece na prática:

```
1  numero_1 = 2
2  numero_2 = 4
3
4  soma = numero_1 + numero_2
5
6  if soma < 10:
7      print("soma não é maior que 10")
8  else:
9      print("soma é maior ou igual a 10")
```

**Código 3.** Operador de comparação

Como podemos ver no **Código 3**, duas variáveis são somadas e esse resultado é atribuído à variável soma. Na linha 6, fazemos uma comparação dessa variável verificando se ela é menor do que 10. Vejamos a seguir, uma operação que faz comparação de igualdade entre duas variáveis:

```
1 soma_1 = 7 + 8
2 soma_2 = 10 + 5
3
4 if soma_1 == soma_2:
5     print("Os resultados são iguais")
6 else:
7     print("Os resultados são diferentes")
```

#### **Código 4.** Operador de comparação

No **Código 4**, criamos duas variáveis e comparamos seus valores, o que pode resultar em dois resultados distintos. Dessa forma, podemos criar estruturas condicionais com o auxílio desses operadores.

## Operadores lógicos

Os operadores lógicos são usados para unir duas ou mais expressões condicionais. Isso é feito por meio de conectivos, como podemos ver na **Tabela 4**.

| Operador | Conceito                                                                            | Exemplo                    |
|----------|-------------------------------------------------------------------------------------|----------------------------|
| and      | Retorna True se todas as condições forem verdadeiras, caso contrário retorna False  | $x > 1$ and $x < 5$        |
| or       | Retorna True se uma das condições for verdadeiras, caso contrário retorna False     | $x > 1$ or $x < 5$         |
| not      | Inverte o resultado: se o resultado da expressão for True, o operador retorna false | not( $x > 1$ and $x < 5$ ) |

**Tabela 4.** Operadores lógicos

Vejamos a seguir, um exemplo que utiliza os operadores lógicos:

```
1  idade_lucas = 21
2  idade_carolina = 19
3
4  # OPERADOR OR
5  if idade_lucas >= 18 or idade_carolina >= 18:
6      print("Pelo menos um dos dois é maior de idade")
7  else:
8      print("Lucas e Carolina não são maiores de idade")
9
10 # OPERADOR AND
11 if idade_lucas >= 18 and idade_carolina >= 18:
12     print("Lucas e Carolina são maiores de idade")
13 else:
14     print("Pelo menos um dos dois não é maior de idade")
```

**Código 5.** Operadores lógicos

No **Código 5** temos um exemplo no qual usamos os operadores `or` e `and`. Com o operador `or`, apenas uma das condições precisa ser verificadas. Já no operador `and`, todas as condições precisam ser satisfeitas. Com a execução do código teremos a impressão das mensagens “Lucas e Carolina são maiores de idade” e "Pelo menos um dos dois é maior de idade".

## Operadores de identidade

Os operadores de identidade servem para a comparação de objetos. Nessa comparação é verificado se eles ocupam a mesma posição na memória, o que significará que se trata do mesmo objeto, como vemos isso na **Tabela 5**.

| Operador | Conceito                                                         | Exemplo           |
|----------|------------------------------------------------------------------|-------------------|
| is       | Retorna True se as variáveis comparadas forem o mesmo objeto     | nome is 'Marcos'  |
| is not   | Retorna True se as variáveis comparadas não forem o mesmo objeto | x is not 'Python' |

**Tabela 5.** Operadores de identidade

Vejamos com mais detalhes o uso dos operadores de identidade:

```
1 time_carlos = 'Botafogo'
2 time_luciano = 'Flamengo'
3 time_fabricia = 'Botafogo'
4
5 if time_carlos is time_luciano:
6     print("time_carlos - time_luciano = mesmo objeto")
7 else:
8     print("time_carlos - time_luciano = objetos diferentes")
9
10 if time_carlos is time_fabricia:
11     print("time_carlos - time_fabricia = mesmo objeto")
12 else:
13     print("time_carlos - time_fabricia = objetos diferentes")
```

#### **Código 6.** Operadores de identidade

No **Código 6** temos um exemplo do operador `is` no Python. Com ele, o que está sendo verificado é se as variáveis apontam para o mesmo objeto. Quando declaramos uma string no Python, o seu valor é guardado num local especial da memória. Assim, quando usamos esse valor repetidas vezes como nas linhas 1 e 3, o Python pode utilizar o mesmo objeto em cada uma delas. Por esse motivo `is` dirá que `time_carlos` e `time_fabricia` apontam para o mesmo objeto, enquanto que `time_luciano` aponta para um objeto diferente.

[Neste artigo](#), podemos ver strings em mais detalhes.

## Operadores de associação

Os operadores de associação são utilizados para verificar se uma sequência contém um objeto. Na **Tabela 6** temos os exemplos.

| Operador | Conceito                                                   | Exemplo    |
|----------|------------------------------------------------------------|------------|
| in       | Retorna True caso o valor seja encontrado na sequência     | 2 in x     |
| not in   | Retorna True caso o valor não seja encontrado na sequência | 2 not in x |

**Tabela 6.** Operadores de associação

No **Código 7**, vamos ver um exemplo usando os operadores de associação.

```
1  frutas = ["banana", "laranja", "uva", "ameixa"]
2
3  fruta_1 = "ameixa"
4  fruta_2 = "melancia"
5
6  print(fruta_1 in frutas) # True
7  print(fruta_2 in frutas) # False
```

**Código 7.** Operadores de associação

O exemplo acima verifica a existência de variáveis dentro de uma lista. Na linha 6 estamos checando se `fruta_1` está contido em `frutas`. Da mesma forma, fazemos isso na linha 7, só que dessa vez com `fruta_2`.

## Conclusão

Ao usar os operadores no Python, podemos atribuir valores a variáveis, criar condições, fazer comparações etc. Essas características fazem parte dos princípios básicos para montar algoritmos nessa linguagem. É importante dominar esses conceitos sobre os operadores, pois eles são amplamente usados no dia a dia do desenvolvedor.